



DandelionDB

File Format Design Specification

Embedded Vector Database for Edge Devices

File is the most important design of this whole project. While the search insert deletes are already invented and optimised we can optimize the file structure for speed and accuracy.

File mainly contains variable sized segments

First the file contains header

Then it contains a fixed size segment index and after that everything is segment



Header Block

Magic Number (Unique identifier of the file)
Version (for backward compatibility)
Checksum Enabled (to check if checksum is enabled for this database managed by scheduler)

Segment Index Block (Fixed Sized, created new when filled up)

Segment header names and example	
Segment ID (4 byte) 32 bit	00000000 00000000 00000000 00000001
Offset (8byte) 64bit	1011010011100101010110110010111010011010 10111001001101011011010101
Total Size (8 byte) 64 bit	1011010011100101010110110010111010011010 10111001001101011011010101
Data Type + Flags (1 Byte) (4 bit for datatype , 4 bit for flag)	00000000
Table number (2 byte) 16bit	00000000 00000000
Column Number (1 byte)	00000000
Row Start Number (8 byte)	00 00000000000000000000000000000000

Important note about segments

Segments are **variable sized blocks** which can scale from **1MB to 256MB**. This size is decided by **Segment Scheduler Engine** during runtime which schedules segment sizes based on table data incoming rate which means it starts from 1MB and goes up to

256 MB if data constantly comes. This is also responsible for managing Dynamic text size

Flags

Bit	Meaning	0	1
Bit 0	Integrity	Invalid / checksum failure	Valid / checksum OK
Bit 1	Capacity	Has available capacity	Full
Bit 2	Residual space	No unusable space	Unusable leftover space exists
Bit 3	Text completion	Text boundary is complete	Text continues into another segment

Checksum is stored in the end of segment

If data arrives

Checksum is removed and new checksum is added in the last

Data Types covered

Datatype + size it can have	Identifier
text (Dynamic)	0001
int32 (32 bit int)	0010
int64 (64 bit int)	0011
uint32 (32 bit unsigned int)	0100
uint64 (64 bit unsigned int)	0101
fp16 (16 bit floating point)	0110
fp32 (32 bit floating point)	0111
fp64 (64 bit floating point)	1000
fp128 (128 bit floating point)	1001
DateTime (64bit)	1010
vector<type,Size>	1011
bool	1100

bytes	1101
Index of a table	1110
Table Schema Declaration	0000
Segment Index Declaration	1111

Table Schema Declaration Segment

Tables are declared in the first Segment of the File when First Table is written
 So its better for optimization if all the required tables are written in one go
 Although we can optimise the whole file with **Optimize function**

Each table gets a number which is used in the Segment Index

A Rough Table might look like this

```
Table UserDocs {
    User_Id int64,
    User_documents text,
    Document_vector vector<f32,764>,
    Created_at DateTime
}
```

```
Table UserChats {
    User_Id int64,
    User_Chats text,
    Chat_vector vector<f32,764>,
    Sent_time DateTime
}
```

The tables roughly takes $64 + 0(\text{variable}) + 24448 + 64 = 24576$ bits each for storing one row

The vectors store differently than the other variables
 Therefore we store each data type in their own segments for optimizing speed and latency for fetching.

Rough Image of Segments

